

What's New in C# 14?

Interview Questions & Answers

Ships with .NET 10 LTS (November 2025) | Visual Studio 2026

1. Overview

Q: What is C# 14 and when was it released?

C# 14 shipped with .NET 10 LTS on November 11, 2025, alongside Visual Studio 2026. .NET 10 is supported until November 2028. To use C# 14, target net10.0 in your .csproj — it is enabled by default with no extra configuration needed.

Q: What are the headline new features of C# 14?

The main features are: (1) Extension members (blocks) — the biggest addition, (2) the field keyword for properties, (3) Partial constructors and events, (4) Null-conditional assignment operator (`?. =`), (5) Overloadable compound assignment operators, (6) `ref/in/out` in lambdas without explicit types, (7) `nameof` for unbound generics, (8) Implicit `Span` / `ReadOnlySpan` conversions.

2. Extension Members (Blocks) — Headline Feature

Q: What are extension members in C# 14 and how are they different from extension methods?

Before C# 14, extension methods (using the `this` keyword) only allowed adding methods to types. C# 14 introduces extension blocks that let you add properties, operators, static members, and indexers to any type — including sealed classes, structs, and built-in types — without modifying the original source.

```
// C# 14 extension block syntax: public static class StringExtensions {  
    extension(string s) { // Extension PROPERTY – not possible before C# 14! public bool  
        IsEmail => s.Contains('@') && s.Contains('.'); // Extension METHOD (as before, but  
        cleaner) public string Truncate(int max) => s.Length <= max ? s : s[..max] + "..."; //  
    Extension STATIC method public static string Repeat(string value, int times) =>  
        string.Concat(Enumerable.Repeat(value, times)); } } // Usage: string email =  
"user@example.com"; bool valid = email.IsEmail; // extension property string short =  
email.Truncate(10); // extension method string rep = String.Repeat("ha", 3); //  
extension static method
```

Why it matters: You can now extend types you don't own (string, int, sealed 3rd-party classes) with properties and operators — a feature developers have wanted since C# 3.

Q: Can you add extension operators with extension blocks?

Yes! This is one of the most powerful additions. You can overload operators for types you don't own, enabling expressive arithmetic or comparison syntax on custom and built-in types.

```
public static class MoneyExtensions { extension(decimal amount) { // Add two money  
    values with currency symbol awareness public static decimal operator +(decimal a,  
    decimal b) => a + b; } }
```

Q: Can extension properties have setters?

Instance extension properties can have getters. Setter support in extension blocks depends on whether the extended type supports mutation (e.g., `ref` structs). The primary use case is read-only

computed properties on reference types.

3. The 'field' Keyword for Properties

Q: What problem does the 'field' keyword solve in C# 14?

Before C# 14, adding any logic to a property accessor (validation, notification) forced you to abandon auto-implemented properties and declare a private backing field manually. This added boilerplate. The 'field' keyword gives direct access to the compiler-generated backing field inside get/set accessors.

```
// BEFORE C# 14 – manual backing field required: private string _name; public string
Name { get => _name; set => _name = value ?? throw new
ArgumentNullException(nameof(value)); } // C# 14 – no backing field needed: public
string Name { get => field; set => field = value ?? throw new
ArgumentNullException(nameof(value)); }
```

Why it matters: Reduces boilerplate in model and entity classes. Particularly useful with INotifyPropertyChanged patterns.

Q: Is 'field' a reserved keyword? What if your class already has a member called 'field'?

No, 'field' is a contextual keyword — it only has special meaning inside property accessors. However, if your class already has a member (field or property) named 'field', it will conflict. The new keyword takes precedence inside accessors, shadowing the member. Best practice: avoid naming members 'field' to prevent confusion.

Q: Can you use 'field' with init-only properties?

Yes. The 'field' keyword works with get, set, and init accessors, making it fully compatible with immutable record-style initialization patterns.

```
public string Email { get => field; init => field = value?.ToLowerInvariant() ?? throw
new ArgumentNullException(nameof(value)); }
```

4. Partial Constructors and Events

Q: What are partial constructors and partial events in C# 14?

C# 14 extends the partial keyword to constructors and events, not just methods and properties. This is especially useful in source-generated code: one part of a partial class is machine-generated, the other is hand-written. A partial constructor lets generated code declare the constructor signature while user code provides the implementation.

```
// Generated part (auto-generated file): public partial class OrderService { public
partial OrderService(ILogger logger); public partial event EventHandler? OrderPlaced; }
// User-written part: public partial class OrderService { public partial
OrderService(ILogger logger) { _logger = logger; _logger.LogInformation("OrderService
initialized"); } public partial event EventHandler? OrderPlaced { add { /* custom
subscription logic */ } remove { /* custom removal logic */ } } }
```

Why it matters: Critical for Source Generator scenarios where generated code needs to hook into constructors and events without forcing users to modify generated files.

5. Null-Conditional Assignment Operator (?.=)

Q: What is the null-conditional assignment operator introduced in C# 14?

The ?.= operator assigns a value to a property or field only if the receiver is not null. It combines the null-conditional ?. with assignment, eliminating a common null-check boilerplate pattern.

```
// BEFORE C# 14: if (user != null) user.LastLoginDate = DateTime.UtcNow; // C# 14:
user?.LastLoginDate = DateTime.UtcNow; // Works with nested paths too:
order?.Customer?.Address?.LastUpdated = DateTime.UtcNow;
```

Why it matters: Cleans up event-handler patterns, cache invalidation, and optional object mutation — fewer null checks, less nesting.

Q: Does `?.=` work with value types?

The receiver must be a nullable reference type (or nullable value type via `Nullable`). The assigned value can be any compatible type. If the receiver is null, the entire assignment is skipped with no exception.

6. Overloadable Compound Assignment Operators

Q: What compound assignment operators can now be overloaded in C# 14?

C# 14 allows user-defined types to overload compound assignment operators (`+=`, `-=`, `*=`, `/=`, etc.) directly. Previously, overloading `+` implicitly gave you `+=`, but the behaviour wasn't fully customizable. Now you can define optimized in-place mutation semantics for your types.

```
public struct Vector2D { public float X, Y; public static Vector2D operator +(Vector2D
a, Vector2D b) => new Vector2D { X = a.X + b.X, Y = a.Y + b.Y }; // C# 14: explicit +=
overload for in-place performance public static Vector2D operator +=(Vector2D a,
Vector2D b) { a.X += b.X; a.Y += b.Y; return a; } }
```

Why it matters: Enables performance-optimised math and DSL types (matrices, vectors, money) with natural assignment syntax.

7. `ref` / `in` / `out` Parameters in Lambdas Without Explicit Types

Q: What improved about `ref/in/out` parameters in lambdas in C# 14?

Before C# 14, using `ref`, `in`, or `out` in a lambda required explicitly stating the parameter type. In C# 14, the compiler can infer the type from context, making lambda syntax more concise when the type is obvious.

```
// BEFORE C# 14 – explicit type required: Delegate d = (ref int x) => x *= 2; // C# 14 –
type inferred: Delegate d = (ref x) => x *= 2; // Useful in high-performance scenarios:
list.ForEachRef((ref Item item) => item.Process()); // C# 14 – shorter:
list.ForEachRef((ref item) => item.Process());
```

Why it matters: Reduces noise when writing performance-critical code that passes values by reference.

8. `nameof` for Unbound Generic Types

Q: What changed with `nameof` and unbound generics in C# 14?

In C# 13 and earlier, `nameof` required a closed generic type (e.g., `nameof(List)` gave `'List'`). C# 14 allows unbound generics: `nameof(List<>)` returns `'List'`. This is cleaner for logging, diagnostics, and reflection where you don't have or care about a specific type argument.

```
// BEFORE C# 14: string name1 = nameof(List); // works but awkward – need a type arg
string name2 = nameof(Dictionary); // verbose // C# 14: string name1 = nameof(List<>);
// "List" string name2 = nameof(Dictionary<,>); // "Dictionary" string name3 =
nameof(IEnumerable<>); // "IEnumerable" // Useful in generic logging utilities: ILogger
logger = loggerFactory.CreateLogger(nameof(Repository<>));
```

Why it matters: Eliminates dummy type arguments in generic utility code — cleaner logs and error messages.

9. Implicit Span and ReadOnlySpan Conversions

Q: What implicit conversions were added for Span in C# 14?

C# 14 adds implicit conversions from T[] (array) to Span and ReadOnlySpan, and from Span to ReadOnlySpan. Previously these required explicit casts or AsSpan(). This makes high-performance APIs easier to call without boilerplate conversions.

```
// BEFORE C# 14 – explicit conversion needed: byte[] data = new byte[256]; ReadOnlySpan
span = (ReadOnlySpan)data; // explicit Process(data.AsSpan()); // extension method //
C# 14 – implicit conversion: byte[] data = new byte[256]; ReadOnlySpan span = data; //
implicit! Process(data); // array passes directly as Span static void
Process(ReadOnlySpan buffer) { /* ... */ }
```

Why it matters: Reduces friction when working with Span-based APIs (System.IO.Pipelines, System.Text, cryptography). Less boilerplate, same performance.

Q: Does this affect performance? Any risks?

No performance impact — the conversion is zero-copy. The only risk is accidentally passing an array where a ReadOnlySpan was intended with mutation expectations — but ReadOnlySpan prevents mutation at compile time, so this is safe.

10. File-Based Apps (No .csproj Required)

Q: What are file-based apps in .NET 10 / C# 14?

You can now run a single .cs file directly with 'dotnet run' — no .csproj, no solution, no bin/obj folders. Dependencies can be declared at the top of the file with a special comment directive. This makes C# practical for scripts, quick prototypes, and simple microservices.

```
#!/usr/bin/env dotnet-script // Newtonsoft.Json using Newtonsoft.Json; var obj = new {
Name = "World", Version = 14 }; Console.WriteLine(JsonConvert.SerializeObject(obj)); //
Run with: dotnet run MyScript.cs
```

Why it matters: C# is now competitive with Python scripts for quick tasks. No project overhead for simple automation.

11. Quick Revision: All C# 14 Features at a Glance

field keyword: Access compiler backing field in property accessors — no manual private field needed

Extension blocks: Add properties, operators, static members to any type (biggest feature)

Partial constructors: Partial keyword now works on constructors (great for Source Generators)

Partial events: Partial keyword now works on events

Null-conditional assignment ?.=: Assign to property/field only if receiver is not null

Compound operator overloads: Overload +=, -= etc. directly on custom types

Lambda ref/in/out inference: Omit type in lambda when using ref/in/out — compiler infers it

nameof(List<>): nameof now accepts unbound generic types

Implicit Span conversion: T[] converts implicitly to Span / ReadOnlySpan

File-based apps: Run a single .cs file with dotnet run — no project file needed